

OBSERVACIONES DE LA PRÁCTICA

Estudiante 1 Cod XXXX

Jimmy Alejandro Mujica Delgado Cod 202224528

Samuel Esteban Diaz Cod 202522112

Máquina 1	
Procesador	12th Gen Intel(R) Core(TM) i5-1235U
Memoria RAM (GB)	8.0
Sistema Operativo	Windows 11 Home Single Language - 64 bits

Tabla 1. Especificaciones de la máquina para ejecutar las pruebas de rendimiento.

Resultados

Carga de Catálogo LINEAR PROBING

Factor de Carga (PROBING)	Consumo de Datos [kB]	Tiempo de Ejecución Real @LP [ms]
0.1	724193.76	8563.34
0.5	425875.13	7417.55
0.7	404349.61	6574.28
0.9	393285.16	7100.05

Tabla 2. Comparación de consumo de datos y tiempo de ejecución para carga de catálogo con el índice por categorías utilizando LINEAR PROBING

Carga de Catálogo SEPARATE CHAINING

Factor de Carga (CHAINING)	Consumo de Datos [kB]	Tiempo de Ejecución Real @SC [ms]
2.00	409415.80	5106.29
4.00	401309.96	6025.89
6.00	398060.24	8109.26
8.00	395228.30	6330.62

Tabla 3. Comparación de consumo de datos y tiempo de ejecución para carga de catálogo con el índice por categorías utilizando SEPARATE CHAINING

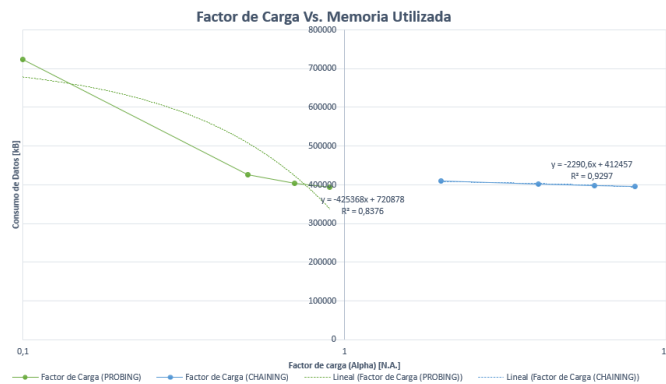
Gráficas

La gráfica generada por los resultados de las pruebas de rendimiento.

- Comparación de memoria y tiempo de ejecución para LINEAR PROBING y SEPARATE CHAINING



- Comparación de factor de carga y memoria para LINEAR PROBING y SEPARATE CHAINING



Preguntas de análisis

1. ¿Por qué en la función **getTime()** se utiliza **time.perf_counter()** en vez de otras funciones como **time.process_time()**?

Perf_counter() mide el tiempo real transcurrido, con alta precisión e incluyendo la lectura de archivos. **Process_time()** solo cuenta el tiempo de CPU y deja por fuera las esperas, así que no refleja lo que realmente carga.

2. ¿Por qué son importantes las funciones **start()** y **stop()** de la librería **tracemalloc**?

Start() activa el registro de la memoria que usa Python, sin el no se puede medir nada, **stop()** lo apaga y libera su memoria, para que el rastreo no siga haciéndose mas lento el resto del programa.

3. ¿Por qué no se puede medir paralelamente el **uso de memoria** y el **tiempo de ejecución** de las operaciones?

Porque **tracemalloc** hace más lenta la ejecución mientras registra la memoria. En las pruebas, la carga tardó unos 30.000 ms con **tracemalloc** y unos 6.500 ms sin él, así que el tiempo real se mide en una corrida aparte.

4. Dado el número de elementos de los archivos (large), ¿Cuál sería el factor de carga para estos índices según su mecanismo de colisión?

Con 10.000 libros, 5.841 autores y 34.252 tags, los índices quedaron con factores entre 0,08 y 0,70 en probing (límite 0,7) y entre 0,47 y 4,0 en chaining (límite 4). En probing conviene un factor menor que 1, alrededor de 0,5 a 0,7. En chaining puede ser mayor que 1, y los mejores tiempos se dieron con 2 a 4.

5. ¿Qué cambios percibe en el tiempo de ejecución al modificar el factor de carga máximo?

En probing, el más lento fue 0,1 (8.563 ms), el mejor 0,7 (6.574 ms), y con 0,9 volvió a subir por las colisiones. En chaining, el tiempo tiende a subir con el factor (de 5.106 ms con 2 a 8.109 ms con 6), porque las listas se alargan.

6. ¿Qué cambios percibe en el consumo de memoria al modificar el factor de carga máximo?

En probing, la memoria baja mucho al subir el factor (de 724.194 kB a 393.285 kB), porque hay menos casillas vacías. En chaining casi no cambia (de 409.416 kB a 395.228 kB), porque cada elemento ocupa un nodo sin importar el factor.

7. ¿Qué cambios percibe en el tiempo de ejecución al modificar el esquema de colisiones? Si los percibe, describa las diferencias y argumente su respuesta.

Chaining con factor 2 fue el más rápido (5.106 ms frente a 6.574 ms, el mejor de probing), porque solo recorre la lista de su casilla, mientras que probing revisa casillas vecinas al haber colisiones. Con factores altos, chaining se vuelve más lento por sus listas largas.

8. ¿Qué cambios percibe en el consumo de memoria al modificar el esquema de colisiones? Si los percibe, describa las diferencias y argumente su respuesta.

Con factores altos, los dos usan memoria parecida (unos 395.000 kB). La diferencia es que probing desperdicia mucha memoria con factores bajos (724.194 kB con 0,1), mientras que chaining se mantiene estable porque solo crea nodos para los elementos que existen.